# Vehicle Rental SaaS — Frontend Build Prompt

Production-grade React + Ant Design implementation prompt — explicitly designed to prevent vibe coding

## OBJECTIVE

Build a production-grade multi-tenant SaaS frontend for businesses that rent cars, bikes and scooters. The system manages vehicles, bookings, customers, payments, deposits, contracts, maintenance, expenses, staff, reports, online booking and company settings.

CRITICAL RULE: DO NOT VIBE CODE. Inspect the existing repository before changing anything. Establish architecture first. Reuse existing code when correct. Do not generate random files, giant components, duplicated UI, fake buttons, or hardcoded repeated data. Keep UI, API logic, state and domain logic separated.

## FRONTEND STACK

Use React, TypeScript, Vite, Ant Design (antd), @ant-design/icons, @ant-design/charts, Ant Design Design Tokens, React Router, TanStack Query, Axios, Zustand, React Hook Form, Zod, @hookform/resolvers and date-fns.

Ant Design is the primary and only UI component library. Do not introduce Material UI, Chakra, Bootstrap, shadcn/ui or another design system. Tailwind is not required. Use Ant Design theme tokens and scoped CSS only where necessary.

## SETUP

If no frontend exists, initialize with:
npm create vite@latest . -- --template react-ts

Install:
npm install antd @ant-design/icons @ant-design/charts
npm install react-router-dom
npm install @tanstack/react-query axios
npm install zustand
npm install react-hook-form zod @hookform/resolvers
npm install date-fns

Configure TypeScript strict mode, ESLint, Prettier, @/* aliases, environment variables, API configuration, error boundaries, global error handling, loading states, empty states, responsive layout, and light/dark Ant Design themes. Never put secrets in frontend source.

## PROJECT ARCHITECTURE

Use a feature-oriented structure:

```
src/
app/ router, providers, theme, layouts
components/ common, data-display, forms, feedback, navigation, overlays
features/ auth, dashboard, bookings, vehicles, customers, payments, contracts, maintenance, expenses, reports,
marketing, staff, notifications, online-booking, subscription, settings
hooks/
lib/ api, auth, permissions, formatting
store/
types/
utils/
assets/
```

Keep feature-specific code inside its feature. Do not create one giant components directory containing business logic.

## APPLICATION SHELL

Build a reusable Ant Design SaaS shell: Layout, Sider, Header, Content, Breadcrumb, responsive/collapsed sidebar, mobile navigation, user menu, notification center, global search, company/tenant selector and theme switcher. Navigation must be permission-aware.

## SIDEBAR + SUBMENUS

Dashboard

```
Operations → Bookings, Calendar, Pickup & Return, Active Rentals, Availability
Vehicles → All Vehicles, Add Vehicle, Categories, Documents, Maintenance, Locations
```

```
Customers → All Customers, Verification, Rental History, Blacklist
Payments → Transactions, Pending Payments, Deposits, Refunds, Invoices
Contracts → All Contracts, Templates, Signatures, Expiring Contracts
Expenses → All Expenses, Fuel, Maintenance Costs, Other Expenses, Categories
Reports → Revenue, Bookings, Fleet Utilization, Customers, Payments, Expenses, Profit & Loss
Marketing → Coupons, Discounts, Campaigns, Referral Codes
Staff → Staff, Roles & Permissions, Activity Logs
Notifications → Notification Center, Templates, Automation Rules
Online Booking → Booking Page, Pricing Rules, Availability Rules, Website Settings
Subscription → Plan, Billing, Usage, Invoices
Settings → Company, Locations, Business Hours, Rental Rules, Pricing, Tax, Payment Methods, Integrations, Branding,
Security
```

## REUSABLE COMPONENTS

```
Create and reuse: AppLayout, AppSidebar, AppHeader, Breadcrumbs, PageHeader, PageContainer, SearchInput, FilterBar,
DataTable, DataTableToolbar, ColumnSettings, StatusBadge, CurrencyDisplay, DateDisplay, DateRangePicker, EmptyState,
ErrorState, LoadingSkeleton, ConfirmModal, FormDrawer, DetailDrawer, FileUploader, ImageUploader, DocumentViewer,
ImageGallery, ActivityTimeline, KPIWidget, TrendWidget, ChartCard, StatsGrid, VehicleCard, CustomerCard, BookingCard,
PaymentCard, PermissionGate and ProtectedRoute.
```

Do not create slightly different copies of existing components.

## DASHBOARD

KPI widgets: Total Revenue, Today's Revenue, Active Rentals, Today's Bookings, Available Vehicles, Rented Vehicles, Pending Payments, Vehicles in Maintenance.

Charts: Revenue Trend, Booking Trend, Fleet Utilization, Revenue by Vehicle Category, Payment Status.

Operational widgets: Today's Pickups, Today's Returns, Overdue Returns, Vehicles Needing Service, Expiring Documents, Recent Bookings, Recent Payments.

Quick actions: New Booking, Add Vehicle, Add Customer, Record Payment, Start Rental, Complete Return.

## OPERATIONS

Bookings: searchable/filterable table, status tabs, dates, customer, vehicle, location, payment status, sorting, pagination, column settings and export. Details include customer, vehicle, dates, price, deposit, payments, contract, documents, notes and activity. Actions: confirm, edit, cancel, payment, pickup, return, extend.

Calendar: day/week/month, vehicle/resource view, booking blocks, availability and conflict indicators.

Pickup & Return: identity/license check, odometer, fuel, damage photos, deposit, signature and completion.

Active Rentals: current rentals, due time, overdue indicator, customer contact and extension.

Availability: date/location/category filters, fleet grid and conflict indicators.

## VEHICLES

All Vehicles: image, plate, make/model, type, location, status, daily price and next service.

Vehicle Details: overview, gallery, specifications, pricing, availability calendar, current rental, documents, maintenance, expenses, damage history and activity.

Add/Edit Vehicle: multi-step form for basic information, pricing, documents, photos, location and rental settings.

Also build Categories, Documents, Maintenance and Locations pages.

## CUSTOMERS

Build All Customers, Customer Details, Verification, Rental History and Blacklist. Customer details should show profile, identity/license documents, rentals, payments, deposits, notes and activity. Verification must support approve, reject and correction requests.

## PAYMENTS + CONTRACTS

Payments: Transactions, Pending Payments, Deposits, Refunds and Invoices. Show booking, customer, amount, method, gateway, status, timestamps, reference and refund information.

Contracts: All Contracts, Templates, Signatures and Expiring Contracts. Contract templates should support reusable variables and a maintainable editor.

## MAINTENANCE + EXPENSES

Maintenance: service queue, schedules, work orders, history, vendors, odometer and costs.

Expenses: all expenses, categories, fuel, maintenance costs, other costs, vendors, receipts and vehicle association.

## REPORTS

Create Revenue, Bookings, Fleet Utilization, Customers, Payments, Expenses and Profit & Loss reports. Each report needs date range, filters, KPI summary, chart, table and export action.

## ONLINE BOOKING PORTAL

Create customer-facing Home, Vehicle Search, Vehicle Details, Checkout, Booking Confirmation and Customer Account pages. Include upcoming bookings, rental history and profile. The portal must be responsive and brandable per rental company.

## SETTINGS

Create Company, Locations, Business Hours, Rental Rules, Pricing, Tax, Payment Methods, Integrations, Branding and Security pages. Use Ant Design Tabs or a settings navigation layout where appropriate.

## DRAWERS / SHEETS / MODALS

Use Drawer for quick create/edit vehicle, customer, booking, payment, expense, maintenance task, document upload, quick details and filters. Use Modal for destructive confirmations, cancellation, payment confirmation, status confirmation and small one-step actions. Use full pages for complex workflows.

## STATE + API

Use TanStack Query for server state, caching, mutations and invalidation. Use Zustand only for client state such as sidebar, theme and UI preferences. Do not put server data into Zustand unnecessarily.

Create a centralized Axios client with interceptors and error handling. Keep API functions inside each feature, e.g. features/bookings/api. UI components must not contain raw Axios calls.

## FORMS + VALIDATION

Use React Hook Form for complex forms and Zod for validation. Apply shared schemas to vehicles, customers, bookings, payments, contracts, expenses, staff and settings. Do not duplicate validation rules.

## TYPESCRIPT

Use strict TypeScript. Do not use any, unnecessary assertions, duplicated interfaces or untyped API responses. Define domain types and explicit component props. Fix type errors rather than suppressing them.

## DESIGN SYSTEM

```
Use Ant Design components: Card, Table, Form, Input, Select, DatePicker, Tabs, Drawer, Modal, Dropdown, Tag, Badge, Statistic, Progress, Calendar, Steps, Upload, Timeline, Descriptions, Grid and Flex. Use Design Tokens for colors, typography, spacing, radius, shadows, control heights and dark mode. The result must look like a polished modern B2B SaaS, not a generic template.
```

## RESPONSIVENESS + ACCESSIBILITY

Support desktop, laptop, tablet and mobile. Tables should use horizontal scrolling, responsive columns and filter drawers rather than simply shrinking. Use semantic HTML, keyboard navigation, focus states, form labels, accessible dialogs, meaningful labels, proper contrast and image alt text.

## STATES

Every data-driven page must implement loading, empty, error, success and retry states. Never leave blank screens while loading. Every table needs loading, empty, error, pagination, filtering and responsive behavior.

## CLEAN CODE — STRICT

```
1. Inspect before implementing.
2. Plan page architecture before coding.
3. Reuse components.
```

4. Keep components small and focused.
5. Separate UI, API and business logic.
6. Never duplicate code.
7. Never hardcode repeated data.
8. No giant files.
9. No any.
10. No suppressed TypeScript errors.
11. No fake functionality.
12. No dead buttons.
13. Isolate mock data when the backend is unavailable.
14. Keep naming/imports consistent.
15. Use reusable hooks.
16. Keep feature boundaries clear.
17. Preserve working code unless change is required.

## IMPLEMENTATION ORDER

```
Phase 1: setup, theme, providers, routing, shell, authentication layout and reusable components.
Phase 2: dashboard, vehicles and customers.
Phase 3: bookings, calendar, availability and pickup/return.
Phase 4: payments, deposits, invoices and contracts.
Phase 5: maintenance, expenses and reports.
Phase 6: online booking, marketing, notifications, staff and subscription.
Phase 7: settings, integrations, responsive refinement, accessibility and final cleanup.
```

After every phase: run TypeScript check, lint, formatting, inspect UI, fix errors and only then continue.

## DEFINITION OF DONE

```
All routes exist, sidebar items work, all pages have proper states, reusable components are used, forms validate, API
logic is separated, permissions work, responsive layouts work, light/dark theme works, there are no TypeScript/lint
errors or obvious console errors, no dead buttons remain, and the frontend can connect to the Go + Chi backend without
rewriting its architecture.
```

## FINAL INSTRUCTION TO THE CODING AGENT

START BY INSPECTING THE EXISTING PROJECT. DO NOT GENERATE EVERY PAGE IN ONE UNCONTROLLED PASS. FIRST BUILD THE FOUNDATION, THEN IMPLEMENT EACH PHASE IN ORDER. Before declaring completion, verify the application with the project scripts, fix all errors, and keep the code clean, typed, reusable and maintainable. DO NOT VIBE CODE.